Automatic Question Generation from Study Material

Oct 28, 2026 · @Asil Khalifa

CSPE72 Deep Learning Techniques, Assignment Team 3. Report due Nov 2, 2026; demo on 29–30 Oct 2026.

Team details

Name	Roll No.	Email id	Contact number
Asil Khalifa	106123015	asilkhalifa11@gmail.com	
L Hitesh Reddy	106123073		
P Kamal Nag	106123093		

Team 3, CSPE72 Deep Learning Techniques. Open: Hitesh's and Kamal's email ids and all three contact numbers still need to be filled in.

Problem statement

Teachers and students spend hours writing practice questions by hand. Given study material in any common form (lecture notes as a PDF, a textbook chapter as text, or pasted paragraphs), the system must automatically generate a set of well-formed, answerable, exam-style questions that cover the material in four formats: short answer, multiple choice with plausible distractors, fill in the blank, and true or false. Every question must carry its answer and the source sentence so a learner can verify it.

Formally, given a document D, produce a set of (question, answer, type) triples such that each answer is a span in D, each question is answerable from D, the set covers the main facts of D, and near-duplicates are removed. The core learning task is answer-aware question generation: learn p(question | passage, answer) with a sequence-to-sequence Transformer (T5) fine-tuned on the SQuAD dataset.

Gaps identified

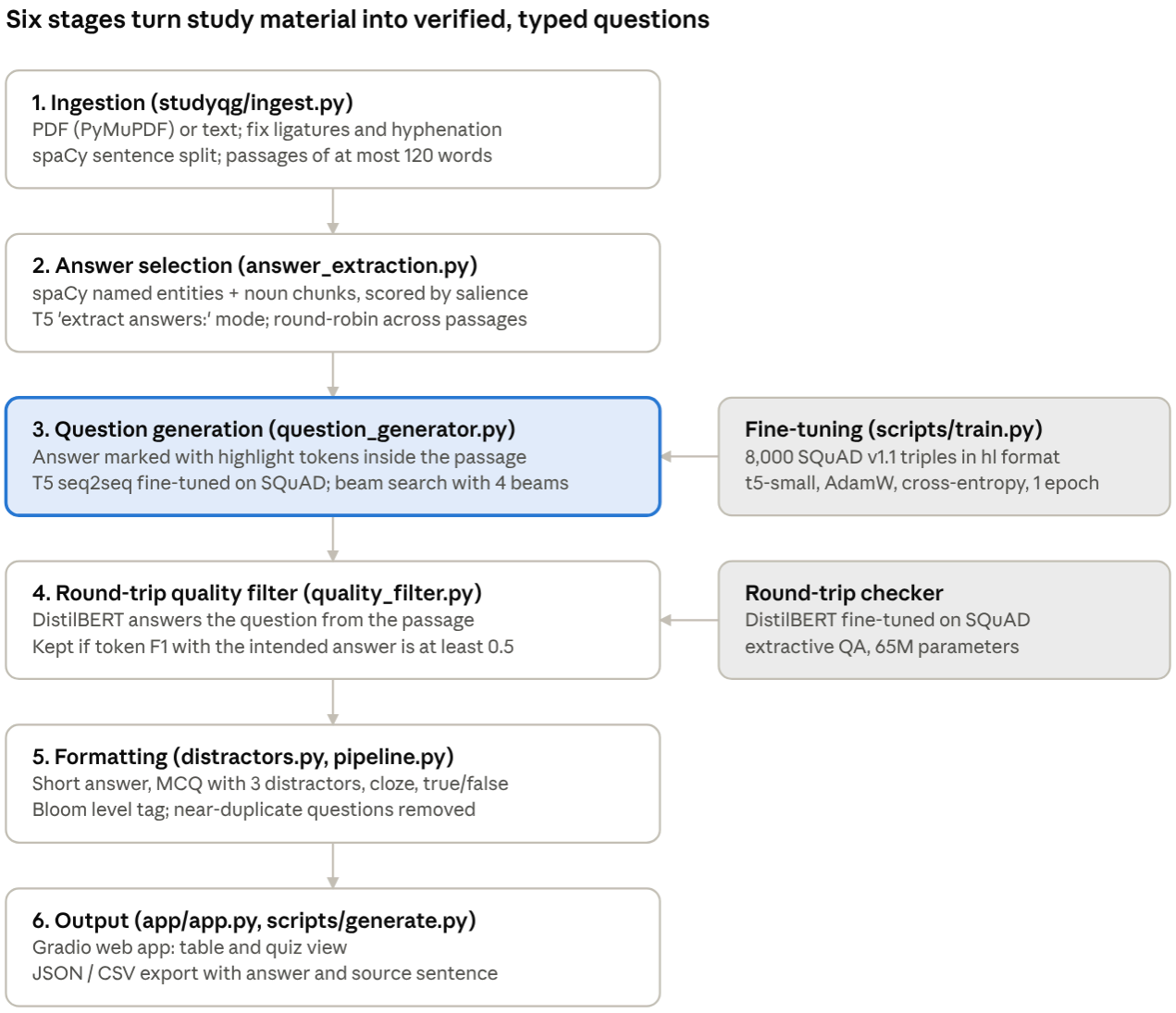
- **Rule-based systems break on real notes.** Classic approaches (Heilman and Smith, 2010) turn declarative sentences into questions with hand-written syntactic rules. They produce stilted questions and fail on the long, complex sentences typical of study

material.

- **Most neural QG work assumes the answer is given.** Models fine-tuned on SQuAD (Du et al., 2017; Dong et al., 2019; Lopez et al., 2020; Ushio et al., 2022) generate a question for a provided answer span. Study material has no marked answers, so a complete system also needs answer selection and coverage of the whole document.
- **No answerability check.** Sequence-to-sequence models hallucinate: they produce fluent questions that cannot be answered from the text, or that leak the answer. Published pipelines rarely verify generated questions automatically.
- **Single question format.** Most systems output only WH-questions. Learners and teachers also need MCQs with sensible distractors, cloze items and true/false statements.
- **Input handling is ignored.** Research code expects clean paragraphs, while real study material arrives as PDFs with headers, page numbers, hyphenated line breaks and ligatures.

Our system closes each gap: a T5 generator fine-tuned on SQuAD, a hybrid answer selector with document-level coverage, a round-trip QA filter, four question formats, and a PDF ingestion stage.

Proposed architecture



StudyQG architecture: 6 inference stages, 2 trained models

The six stages on the left run for every document. The two boxes on the right are the learned components: the T5 generator we fine-tune once on SQuAD, and the DistilBERT question-answering model that checks every generated question. The question generator (stage 3) is the deep learning core of the system.

Algorithmic steps

Training (once, scripts/train.py)

1. Load SQuAD v1.1 and convert each (context, answer, question) triple to highlight format: the answer span inside the context is wrapped in `<hl>` tokens and the task prefix "generate question:" is prepended. 8,000 training and 300 validation examples are sampled with a fixed seed.

2. Load pre-trained t5-small, add `<hl>` as a special token and resize the embedding matrix.
3. For each mini-batch of 8: tokenise the source (up to 384 tokens) and the target question (up to 48 tokens); run the encoder on the source and the decoder on the right-shifted target (teacher forcing); compute token-level cross-entropy against the target.
4. Back-propagate, clip the gradient norm to 1.0 and take an AdamW step (learning rate 3e-4, weight decay 0.01) with 5% linear warm-up followed by linear decay.
5. After each epoch compute the validation loss; save the model and tokenizer.

Inference (per document, studyqg/pipeline.py)

1. Ingest: read the PDF or text, normalise ligatures and quotes, undo hyphenated line breaks, drop lines that are mostly non-alphabetic (page numbers, rules), split into sentences with spaCy and group them into passages of at most 120 words without breaking a sentence or a paragraph.
2. Candidate answers: for every sentence collect named entities (persons, dates, organisations, numbers and so on) and noun chunks with spaCy, score them by type and grammatical role, and add a salience bonus proportional to log(frequency in the document). With the lmqg checkpoint also run T5 in "extract answers:" mode with each sentence highlighted. Deduplicate per passage.
3. Select: pick candidates round-robin across passages (one per sentence first) until 1.8 × N are chosen, so the questions cover the whole document.
4. Generate: highlight each chosen answer inside its passage and run T5 with beam search (4 beams, no repeated 3-grams, up to 64 new tokens). Tidy the output (capitalise, ensure a question mark) and drop near-duplicates (string similarity above 0.9) and questions that contain the answer verbatim.
5. Verify: ask DistilBERT-SQuAD each question with the passage as context; compute token F1 between its answer and the intended answer; keep questions with F1 ⩾ 0.5 and rank by (F1, QA confidence, candidate score). Take the top N and restore document order.
6. Format: cycle through the requested types. MCQ: 3 distractors from same-type spans in the document, WordNet siblings or perturbed numbers, shuffled with the answer. Fill in the blank: the source sentence with the answer blanked. True/false: the source sentence as is, or with the answer swapped for a distractor. Tag a Bloom level from the question wording.
7. Output: table, quiz view, JSON and CSV with the answer, quality score and source sentence of every item.

Dataset used

Dataset	Use	Size	Source
SQuAD v1.1 (Stanford Question Answering Dataset)	Fine-tuning and evaluation of the question generator	87,599 train / 10,570 validation triples from 536 Wikipedia articles; we sample 8,000 train and 300 validation (one per paragraph)	rajpurkar/squad on Hugging Face
Sample study material	Demo inputs	Three passages of about 350 words written by the team: photosynthesis, neural networks, ISRO	`data/sample/` in the repo

SQuAD provides (context paragraph, question, answer span) triples (Rajpurkar et al., 2016). We reverse the task: the model receives the context with the answer highlighted and must produce the question. The same dataset trained the public lmqg checkpoints and the DistilBERT QA model used in the filter, so every learned component shares one annotation convention.

Tools used

Tool	Version	Purpose
Python	3.12	Implementation language
PyTorch (CPU build)	2.14	Model training and inference
Hugging Face Transformers	5.21	T5 and DistilBERT models, tokenizers, beam-search generation
Hugging Face Datasets	5.1	Loading SQuAD v1.1
spaCy with en_core_web_sm	3.8	Sentence splitting, named entities, noun chunks
NLTK with WordNet	3.10	METEOR metric, WordNet distractors
sacrebleu, rouge-score	2.6	BLEU-4 and ROUGE-L metrics
PyMuPDF	1.27	PDF text extraction
Gradio	6.31	Web demo

Tool	Version	Purpose
pandas, matplotlib	3.1, 3.11	Result tables and plots
Git and GitHub		Version control and code submission
Claude Code (Claude Fable 5.1)		AI assistant; see AI tool usage

Hardware: a 16-core CPU laptop with 15 GB RAM and no GPU (Ubuntu under WSL2). Everything in the report, including fine-tuning, was run on it.

Screenshots of the result

Figure 1. The Gradio demo after generating 10 questions from the Neural networks sample (398 words, 4 passages): 66 candidate answers, 18 questions generated, 18 of 18 passed the round-trip filter, mean quality 1.0. It took 157.6 s because the fine-tuning job was running on the same CPU at the time.

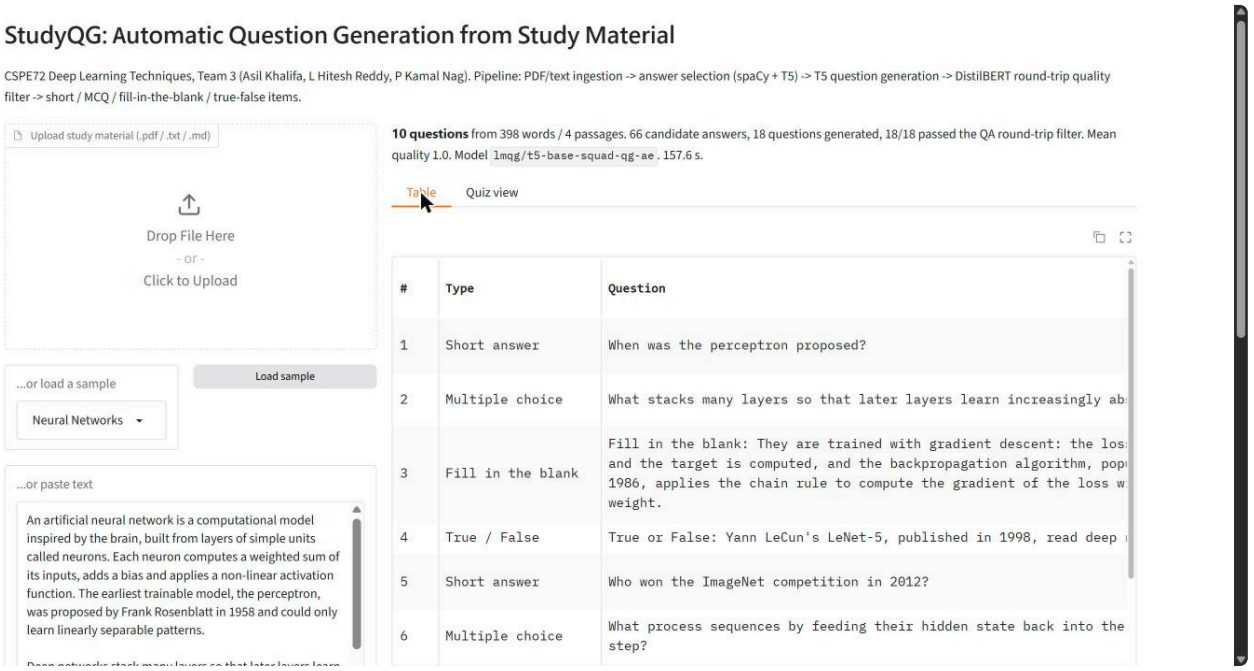


Figure 2. Quiz view with the answer key, the quality score and the source sentence of every item. Q4 is a true/false item built by swapping the verified answer (handwritten digits) for a distractor.

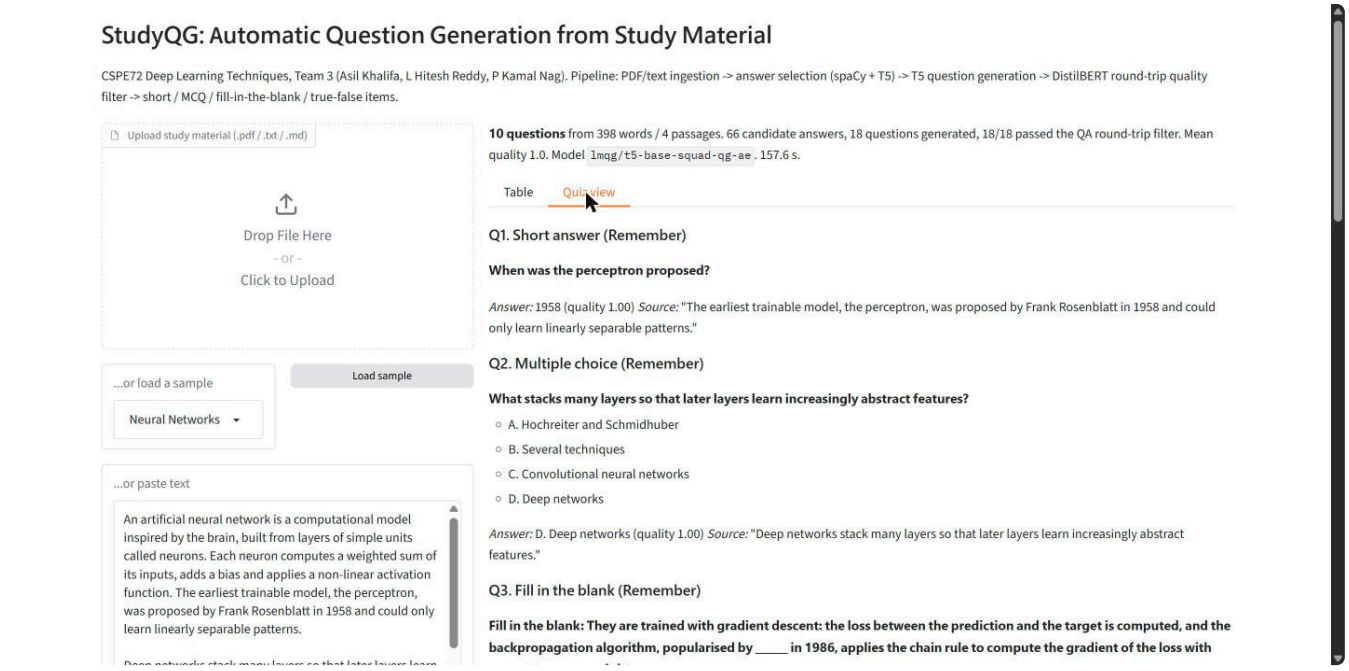


Figure 3. Fine-tuning t5-small on 8,000 SQuAD examples (scripts/train.py, 1,000 steps of batch 8, 49 minutes on the CPU): training loss per 20 steps and validation loss before training and at the end of the epoch (3.66 to 1.68).

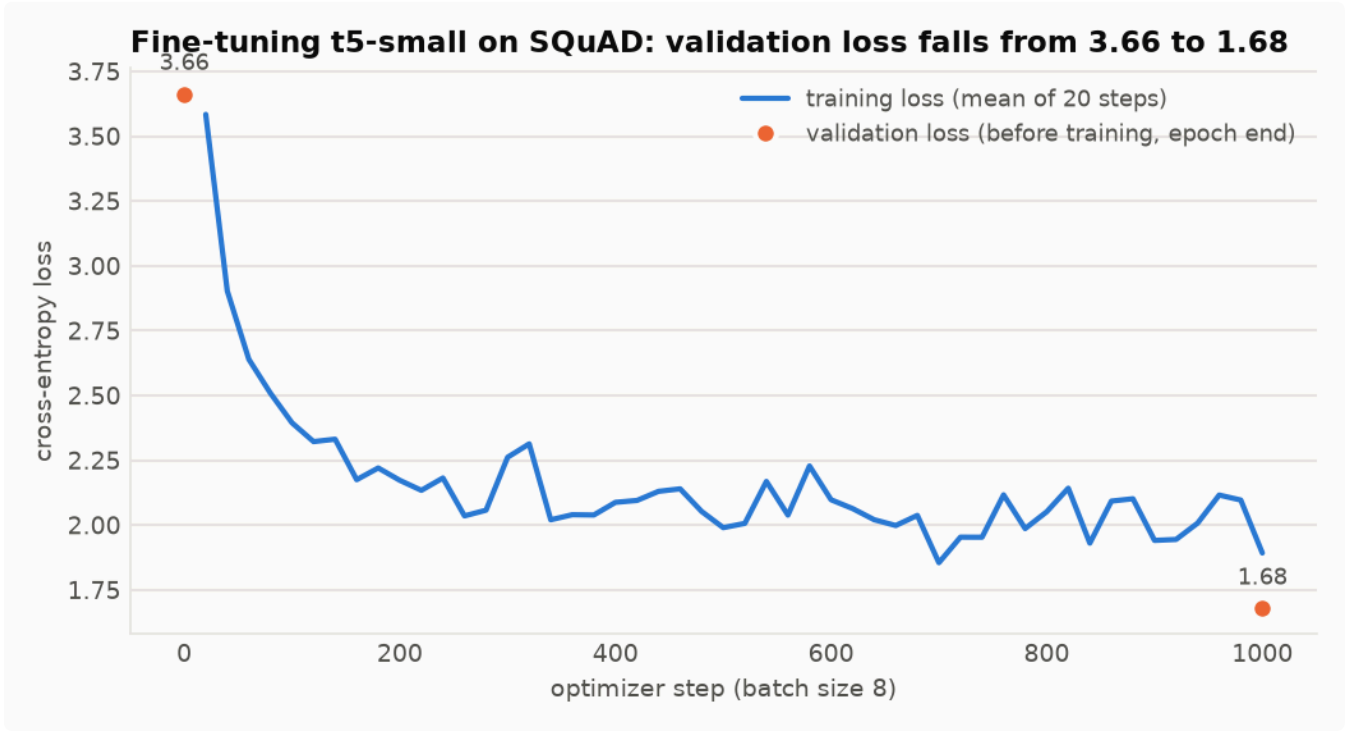
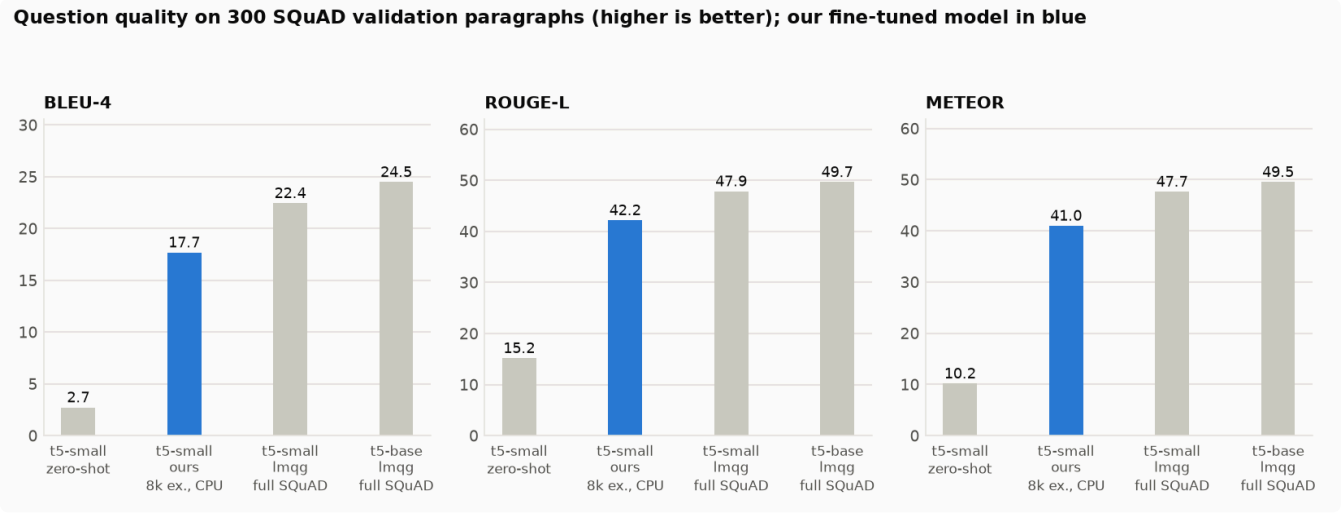


Figure 4. BLEU-4, ROUGE-L and METEOR of the four generators on 300 SQuAD validation paragraphs (scripts/evaluate.py and scripts/plot_results.py); our fine-tuned model in blue.



Analysis and how it differs from existing approaches

Quantitative results. Evaluation on 300 SQuAD v1.1 validation paragraphs (one question each, beam size 4, scripts/evaluate.py). Scores are 0–100, higher is better; timings are per question on the laptop CPU with 8 threads.

Model	Training data	BLEU-4	ROUGE-L	METEOR	Seconds per question
t5-small, zero-shot (no fine-tuning)	none	2.7	15.2	10.2	0.55
t5-small, ours (scripts/train.py)	8,000 SQuAD examples, 1 epoch, 49 min on CPU	17.7	42.2	41.0	0.63
t5-small, lmqg	full SQuAD (87,599 examples), GPU	22.5	47.9	47.7	0.91
t5-base, lmqg (demo default)	full SQuAD (87,599 examples), GPU	24.5	49.7	49.5	2.69

- **Fine-tuning is what makes the model work.** The zero-shot T5 mostly copies a span of the passage ("the Teaching Council") instead of asking a question. After 1,000 optimiser steps on a CPU, the validation loss falls from 3.66 to 1.68 and BLEU-4 rises from 2.7 to 17.7, which closes about 80% of the gap to the lmqg t5-small that was trained on 11 times more data with a GPU.
- **Data and model size buy the rest.** Full-data training adds 4.8 BLEU-4 at the same size; moving to t5-base adds another 2.0 BLEU-4 and 1.8 ROUGE-L at three times the latency. The demo therefore defaults to t5-base and offers t5-small for speed.
- **The scores are in line with the literature.** Du et al. (2017) reported BLEU-4 12.3 on SQuAD. The lmqg t5-base model card reports 26.1 BLEU-4 and 53.3 ROUGE-L on the full development set with its own tokeniser; our 24.5 and 49.7 on a 300-paragraph sample with sacrebleu and rouge-score are consistent with that.

- **Low exact match (1–5%) shows why reference-based metrics undercount.** For the answer "territory east of the Mississippi" the reference asks "What territory was ceded to Britain?" while our model asks "What territory did France cede to Great Britain?": a correct question that scores poorly on BLEU. This is why the system also checks every question with the round-trip QA filter. On the neural-networks sample 18 of 18 generated questions were answerable by DistilBERT with F1 at least 0.5; on the ISRO sample 11 of 15.
- **Failure modes we saw.** The small model sometimes produces an ungrammatical question ("Who was the seniority of Genghis' oldest son Jochi?") or leaks the answer into the question; the pipeline drops questions that contain the answer verbatim and relies on the QA filter for the rest. Distractors for abstract noun phrases are weaker than for names, dates and numbers.

How the system differs from existing approaches

Approach	Needs the answer given?	Checks answerability?	Formats	Input handled
Rule-based transformation (Heilman and Smith, 2010)	No, but rules pick trivial spans	Ranking only	WH-questions	Clean sentences
Seq2seq on SQuAD (Du et al., 2017)	Yes	No	WH-questions	Clean paragraphs
lmqg T5 checkpoints (Ushio et al., 2022)	Yes, or its own extractor	No	WH-questions	Clean paragraphs
Prompting a large hosted LLM	No	No, can hallucinate	Any	Pasted text; the material leaves the machine
StudyQG (ours)	No: hybrid answer selection with document coverage	Yes: DistilBERT round trip with a visible score	Short answer, MCQ, cloze, true/false	PDF or text, offline on a CPU

Comparison with published question-generation models on SQuAD

System	Year	Architecture	Training data	BLEU-4	ROUGE-L	METEOR
Du et al., Learning to Ask	2017	BiLSTM encoder–decoder with attention	full SQuAD	12.3	39.8	16.6
Zhao et al., maxout pointer	2018	gated self-attention seq2seq, paragraph level	full SQuAD	16.4	44.5	20.3
UniLM (Dong et al.)	2019	pre-trained Transformer, unified LM	full SQuAD	22.1	51.1	25.1
ProphetNet (Qi et al.)	2020	pre-trained Transformer, n-stream decoder	full SQuAD	25.0	52.6	26.8
lmqg BART-base (Ushio et al.)	2022	BART encoder–decoder	full SQuAD	24.7	52.7	26.1
lmqg T5-small (Ushio et al.)	2022	T5 encoder–decoder, 60M	full SQuAD	24.4	51.4	25.8
lmqg T5-base (Ushio et al.)	2022	T5 encoder–decoder, 220M	full SQuAD	26.1	53.3	27.0
Ours, t5-small fine-tuned on CPU	2026	T5 encoder–decoder, 60M	8,000 SQuAD examples, 1 epoch	17.7	42.2	41.0

Published rows are the numbers reported in the respective papers and model cards (Du et al. test split, nlg-eval scorers); our row uses 300 paragraphs of the official development set with sacrebleu and rouge-score, so the comparison shows trends rather than exact decimals. The lmqg checkpoints scored with our scorer (24.5 BLEU-4 and 49.7 ROUGE-L for T5-base against 26.1 and 53.3 on the card) give the size of that scorer offset, about 2 to 4 points. Our METEOR is not comparable because NLTK's implementation counts synonyms and stems. Read this way, our one-hour CPU model sits between the 2017 LSTM system and the 2018 paragraph-level system on BLEU-4 and ROUGE-L, and the T5-base checkpoint we ship in the demo is at the level of the strongest published SQuAD question generators.

Why T5 and not another model family. The task is conditional generation of a short question from a long passage with one marked span. Encoder-only models (BERT, DistilBERT) produce one vector per input token and cannot write text, which is why DistilBERT is our checker rather than our generator. Decoder-only models (GPT-2 style) can write but read left to right only, so the passage and the question compete for one context window and the task has to be conveyed by prompt wording. An encoder–decoder reads the passage bidirectionally once and a separate decoder writes the question while attending to every passage token through cross-attention; this family holds the published SQuAD question-generation records (UniLM, ProphetNet, lmqg). Among encoder–decoders we chose T5 because its text-to-text framing makes the task prefix and the `<hl>` highlight token natural and lets one model do both answer extraction

and question generation, because t5-small is small enough to fine-tune on a CPU in 49 minutes while t5-base is still fast enough for a live demo, because public SQuAD checkpoints in exactly our input format exist for a controlled comparison, and because pre-training on C4 gives strong transfer (8,000 examples took BLEU-4 from 2.7 to 17.7). The cost is that a large hosted LLM would write more varied questions; we traded that for a model we can train, measure and run offline.

Code link

GitHub: https://github.com/asil-khalifa/dl-project (public repository with README, code, sample data, results and screenshots).

Setup and run instructions are in the README; the model checkpoint is reproduced by scripts/train.py (about 49 minutes on a CPU) and is not stored in the repository.

Novel contribution

Published question-generation models stop at "answer in, question out". Our contribution is the system around the model that makes it usable on real study material, plus a generator trained by us:

1. **Round-trip verified generation.** Every generated question is answered by an independent extractive QA model, and only questions whose recovered answer matches the intended one (token F1 at least 0.5) are kept. An unreliable generator becomes a filtered one, and each question carries a quality score the user can see. We adapt the round-trip consistency idea of Alberti et al. (2019), originally used to build synthetic QA corpora, into an interactive quality gate for study questions.
2. **Coverage-aware hybrid answer selection.** Candidates come from two sources (spaCy entities and noun chunks with a document-frequency salience score, and T5's own answer-extraction mode) and are picked round-robin across passages, so a 10-question quiz spans the whole chapter instead of its first paragraph.
3. **Four question formats from one generator.** Distractors are mined from same-type spans in the same document (an MCQ about Chandrayaan-3 offers other ISRO missions, not random words), with WordNet and numeric fallbacks; cloze and true/false items reuse the verified source sentence.
4. **Our own fine-tuned generator and a controlled comparison.** We fine-tune t5-small on 8,000 SQuAD examples in 49 minutes on a CPU and measure it against a zero-shot baseline and the full-data lmqg checkpoints with BLEU-4, ROUGE-L and METEOR, which shows how much quality comes from fine-tuning versus model size and data.
5. **End-to-end PDF handling with traceable output.** Every item carries its source sentence and a Bloom level, and the whole pipeline runs on a laptop without a GPU.

Individual contribution

Member	Contribution
Asil Khalifa (106123015)	Pipeline design, T5 question generator and fine-tuning (question_generator.py, scripts/train.py), evaluation and plots, repository and README
L Hitesh Reddy (106123073)	Ingestion and answer selection (ingest.py, answer_extraction.py), sample study material, round-trip QA filter integration, unit tests
P Kamal Nag (106123093)	Distractor generation and question formatting (distractors.py, pipeline.py), Gradio demo app (app/app.py), report and demo presentation

All three members reviewed the code, ran the experiments and co-wrote the report. Open: this split is a proposal; edit the table so it matches how the work was actually shared.

Learning outcome

- How a text-to-text Transformer (T5) is fine-tuned for a new task: data formatting with task prefixes and highlight tokens, tokenisation, teacher forcing, cross-entropy loss, AdamW with warm-up, and checkpointing.
- The difference between training-time and inference-time behaviour of a seq2seq model, and how decoding choices (beam size, repetition constraints) change the output.
- How to evaluate generated text with BLEU-4, ROUGE-L and METEOR, what these metrics reward, and why a reference-free check such as round-trip QA is also needed.
- How to use pre-trained models from the Hugging Face hub (lmqg T5, DistilBERT-SQuAD) as components, and how model size and training data trade off against CPU latency.
- Classical NLP tooling (spaCy NER, noun chunks, WordNet) still matters for answer selection and distractors around a neural model.
- Engineering a complete system: PDF ingestion, chunking, batching for CPU inference, a Gradio interface, reproducible scripts and unit tests.

AI tool usage

Tool	Purpose
Claude Code (Anthropic, model Claude Fable 5.1)	Scaffolded the repository and wrote the first version of the pipeline code, training and evaluation scripts, Gradio app, README, this report and the team's study cheatsheet under the team's direction. The team reviewed, ran and debugged the code and verified every result reported here.
Hugging Face Hub models	Pre-trained T5 (google-t5/t5-small), the lmqg T5 question-generation checkpoints and DistilBERT-SQuAD, used as components of the system.
spaCy, NLTK WordNet	Rule-based NLP components, not generative AI.

No AI tool was used to produce results: all metrics come from runs of scripts/evaluate.py on the SQuAD validation subset, and the screenshots are from the running app.

Prompts issued to the AI agent

All prompts were typed by the team into Claude Code (Anthropic) running in the project folder between 28 Oct and 1 Nov 2026. The agent could read the assignment PDF, run code on the laptop and edit the report. For each prompt we give the text as issued and a refined version that states context, constraints and the acceptance criterion up front; the refined versions are what we would issue next time.

1. Kick-off: build the whole submission

As issued: "do the project for our team (Team 3 asil hitesh and kamal) including all the things required to submit. Also create a cheatsheet document for us to explain things related to the project"

Refined: "You are helping Team 3 of CSPE72 Deep Learning Techniques (Asil Khalifa 106123015, L Hitesh Reddy 106123073, P Kamal Nag 106123093). Our assigned problem is Automatic Question Generation from Study Material; the assignment PDF in this folder lists the deliverables (demo on 29–30 Oct, report and GitHub link on 2 Nov, 14 report sections). Build a complete, demo-able system: a deep learning model that we fine-tune ourselves plus a stronger public baseline, a web demo that accepts a PDF or pasted text, a GitHub-ready repository with README and tests, and a report covering every required

section. Constraint: this laptop has 16 CPU cores and no GPU, so keep training under about an hour and report real measured numbers only. Also write a cheatsheet for the three of us that explains the concepts, the code and likely viva questions in plain language."

2. Divide the work among three members

As issued: "also split the tasks required for this project into 3 (for 3 members) in the cheatsheet - include stuff where we gave our (human) input (eg: research, ideation) and stuff where AI was more useful (eg: imlementation)"

Refined: "In the cheatsheet add a section that divides the project among the three members so that each owns one slice end to end. For every slice separate (a) the human contribution: literature reading, design decisions, threshold choices, testing on real notes, interpretation of results, and (b) the parts where the AI assistant did most of the implementation or drafting. Keep it honest and consistent with the Individual Contribution table in the report."

3. Progress check

As issued: "status?"

Refined: "Give a short status: what is finished, what is still running with an estimated finish time, what is blocked on us, and the links to the report and the cheatsheet."

4. Publish the code

As issued: "push here: https://github.com/asil-khalifa/dl-project"

Refined: "Push the repository to https://github.com/asil-khalifa/dl-project (public, branch main). Update the README clone URL and the report's Code link section to this address, and list what still remains for the submission."

5. Demo preparation

As issued: "how to setup the project for a demo"

Refined: "Write demo setup instructions for two cases: this laptop, where everything is installed, and a fresh laptop that clones the repo. Include pre-downloading the models, how to start the app, a recommended five-minute demo flow with timings, a faster fallback model, and what to show if the live demo fails."

6. Model choice

As issued: "what other techniques did you try apart from T5 transfomer? why did you choose this?"

Refined: "List the alternative model families you considered for question generation (rule-based, RNN seq2seq, BERT-style encoders, GPT-style decoders, BART, prompting a hosted LLM) and explain the trade-offs that led to T5. State clearly which alternatives were actually run and which were only considered."

7. Understanding the quality filter

As issued: "if bert is encoder only and can't generate text how did we use it to do QA check on the questions given by T5"

Refined: "Explain how an encoder-only model such as DistilBERT answers a question without generating text (the start and end span heads), how we compare its span with the intended answer, and give a one-line version we can say in the viva."

8. Understanding the pipeline order

As issued: "so when the doc is first given, T5 and spacy first pick out answers to question? and then T5 generates questions for those answers?"

Refined: "Confirm the order of the stages: how answer spans are chosen (spaCy and the T5 extract-answers mode), how T5 then generates a question for each span, and what changes when our own fine-tuned model is used instead of the public one."

9. Understanding the metrics

As issued: "what are those blue1 rouge l"

Refined: "Explain BLEU-1, BLEU-4, ROUGE-L, METEOR and exact match in one short paragraph each, with an example from our own outputs and our actual scores, and say what each metric cannot measure."

10. Understanding the Bloom tag

As issued: "which stage comes up with the bloom"

Refined: "Which pipeline stage assigns the Bloom level, how exactly is it computed, and how should we describe its limitation?"

11. Final report

As issued: "more clearly why we chose t5 instead of other options. what is advantage. how to explain. as per requirements make a pdf report for the project. Make sure to include analysis section consisting of comparison with existing models for this, Also, have a prompts issued section that contains the prompts given from our side to the agent. Refine these prompts, make them better."

Refined: "Write a clear, viva-ready explanation of why T5 was chosen and what its advantages are. Then produce the final report as a PDF that follows the assignment's section list, with an analysis section comparing our models against published question-

generation systems on SQuAD, and a section listing the prompts we issued to the AI agent together with improved versions. Save the PDF in the repository and push it."

What we learned about prompting. The prompts that worked best named the context (course, team, deadline, hardware), the deliverable and the acceptance criterion in one message. Short follow-up questions were the right way to understand what had been built before the viva. Asking the agent to state what it did not try, and to report only measured numbers, kept the report honest.